

C# / .NET Learning Roadmap

Topic Notes — Section 10: Unit Testing and Debugging

Topics: 10.1 through 10.6

Date: 29 June 2026

Section 10 — Unit Testing and Debugging

This section covers all six roadmap topics: 10.1 (Debugging tools), 10.2 (xUnit basics), 10.3 (Assertions, test lifecycle), 10.4 (Mocking with Moq), 10.5 (TDD concept), 10.6 (Integration testing basics).

Teaching order: 10.1 (debugging) stands alone. 10.2-10.4 build sequentially. 10.5 (TDD) uses everything from 10.2-10.4. 10.6 (integration testing) extends 10.2.

10.1. Debugging Tools (Breakpoints, Watch, Call Stack)

Builds on: Nothing specific -- debugging is applied to any C# code. Works best once you have code to debug from previous sections.

What debugging is and why it matters

Writing code is half the job. The other half is figuring out why it does not do what you expected. The alternatives -- `Console.WriteLine` scattered through code -- are slow, pollute production logs, and get forgotten. A debugger lets you inspect any variable at any point without modifying a single line of code.

Breakpoints -- pausing execution at a specific line

A breakpoint tells the debugger: Stop execution here and hand control to me. Click the grey margin left of any line number. A red circle appears. Press F5 (Start Debugging). When execution reaches that line, it pauses BEFORE executing it.

```
public async Task<decimal> CalculateTotalAsync(int orderId)
{
    // Set breakpoint on the line below:
    var order = await _repo.GetByIdAsync(orderId);
    // Execution pauses here before it runs.
    // You can now inspect: orderId, _repo, and after
    // stepping, the value of order.

    decimal subtotal =
        order!.Items.Sum(i => i.Price * i.Quantity);
    decimal tax = subtotal * 0.18m;
    return subtotal + tax;
}
```

Keyboard shortcuts (Visual Studio):

F5	Start debugging
F9	Toggle breakpoint on current line
F10	Step Over -- run line, pause on next
F11	Step Into -- enter the method called
Shift+F11	Step Out -- run until method returns
F5 again	Continue -- run to next breakpoint
Shift+F5	Stop debugging

Conditional breakpoints -- pause only when condition is true

When a loop processes 10,000 orders, you don't want to pause on every iteration. Right-click any breakpoint → Conditions → enter a C# boolean expression. Execution only pauses when the condition is true.

```
foreach (var order in orders)
{
    var total = CalculateTotal(order);
    // Right-click breakpoint on line above:
    // Conditions: order.Id == 42
    // OR: total > 10000
    // Pauses only on the matching iteration
}
```

Watch window -- inspecting variables

```
// While paused, type these in the Watch window:
orders                // see the full list
orders.Count          // 47
orders[0]             // expand to see all properties
orders[0].Amount      // 999.99
orders.Sum(o => o.Amount) // evaluate any LINQ expression
orders.Any(o => o.IsPaid) // true
DateTime.UtcNow       // evaluate any C# expression

// Other windows:
// Locals    -- all local variables in current scope
// Autos     -- variables near the current line
// Immediate-- run arbitrary C# while paused
```

The Call Stack -- understanding how you got here

The Call Stack shows the chain of method calls that led to the current line. Click any frame to jump to that code with its local variables visible.

```
Call Stack (bottom = outermost, top = current):
[External code]                ASP.NET Core
OrdersController.GetById (id=42)
OrderService.GetByIdAsync (id=42)
OrderRepository.GetByIdAsync (id=42)
AppDbContext.FindAsync (id=42) <-- you are here
```

Exception debugging -- break on throw

By default the debugger only pauses on unhandled exceptions. Configure it to pause at the exact throw site before any catch block runs: Debug → Windows → Exception Settings → check 'Common Language Runtime Exceptions'. Now the debugger pauses showing you exactly which expression caused the exception.

Common debugging scenarios

```
NullReferenceException, don't know what's null:
-> Enable break-on-throw for CLR exceptions
-> Hover over each sub-expression on the failing line

Wrong value in a variable:
```

- > Breakpoint before the calculation
- > Step through (F10) line by line
- > Watch the variable -- find where it goes wrong

Method called with wrong argument:

- > Check the Call Stack -- click the caller frame

Async operation never completes:

- > Debug -> Windows -> Tasks
- > Look for tasks stuck in Waiting state
- > Check for .Result or .Wait() causing deadlock

Key Takeaways for 10.1

- Breakpoints pause execution before a line runs -- F9 to set, F5 to run
- Conditional breakpoints pause only when a C# expression is true
- Watch/Locals/Autos inspect any variable or expression while paused
- Call Stack shows how you got here -- click frames to navigate
- Break-on-throw pauses at the throw site before any catch block
- F10 Step Over, F11 Step Into, Shift+F11 Step Out

Memory aid: The debugger is a time machine. Breakpoints freeze time. The Watch window is a microscope. The Call Stack is the history book showing how you got there.

10.2. xUnit Basics

Builds on: 9.5 (manual DI -- tests use constructor injection to wire fakes), 9.2 (repository -- InMemoryRepository used in tests), 7.3 (async/await -- test methods are async Task).

What is xUnit and why does it exist?

xUnit is the most popular unit testing framework for .NET. It discovers and executes test methods, provides assertions to verify expected behaviour, isolates tests (each test gets a fresh class instance), and runs tests in parallel by default. Unit tests catch regressions, enable safe refactoring, and serve as executable documentation.

Setting up xUnit

```
<!-- Add to your test .csproj: -->
<PackageReference Include="xunit" Version="2.9.*" />
<PackageReference Include="xunit.runner.visualstudio"
    Version="2.8.*" />
<PackageReference Include="Microsoft.NET.Test.Sdk"
    Version="17.11.*" />
<ProjectReference Include="..\MyApp\MyApp.csproj" />

# Run tests:
dotnet test
dotnet test --filter "FullyQualifiedName~OrderServiceTests"
dotnet test --verbosity normal
```

Your first xUnit test -- AAA pattern

```
using Xunit;

public class CalculatorTests
{
    // [Fact] -- a single test with no parameters:
    [Fact]
    public void Add_TwoPositiveNumbers_ReturnsSum()
    {
        // Arrange -- set up the scenario:
        var calculator = new Calculator();
        int a = 5, b = 3;

        // Act -- perform the operation being tested:
        int result = calculator.Add(a, b);

        // Assert -- verify the expected outcome:
        Assert.Equal(8, result);
    }
}
```

The Arrange-Act-Assert (AAA) pattern is the universal structure for unit tests. Every test has exactly these three phases, separated by blank lines.

Test naming convention

Test names are documentation. A failing test named Test1 tells you nothing. Convention:

MethodUnderTest_Scenario_ExpectedBehavior

```
// Good test names -- readable as specifications:
[Fact] void Add_TwoPositives_ReturnsSum() { }
[Fact] void Divide_ByZero_ThrowsException() { }
[Fact] void GetById_NonExistentId_ReturnsNull() { }
[Fact] async Task CreateOrder_ValidDto_SavesOrder() { }
[Fact] async Task Cancel_ShippedOrder_ThrowsInvalidOp() { }
```

[Theory] and [InlineData] -- data-driven tests

```
public class OrderValidatorTests
{
    private readonly OrderValidator _validator = new();

    // [Theory] runs 4 times with different inputs:
    [Theory]
    [InlineData(0)]
    [InlineData(-1)]
    [InlineData(-100.5)]
    [InlineData(-0.01)]
    public void Validate_NonPositiveAmount_ReturnsError(
        decimal amount)
    {
        var dto = new CreateOrderDto
        { Amount = amount, CustomerName = "X" };
        var result = _validator.Validate(dto);
        Assert.False(result.IsValid);
    }
}
```

```

        Assert.Contains("Amount", result.Errors[0]);
    }
}

// [MemberData] for complex test data:
public static IEnumerable<object[]> InvalidOrders =>
    new List<object[]>
    {
        new object[]
        {
            new CreateOrderDto
                { Amount = -1, CustomerName = "X" },
            "Amount"
        },
        new object[]
        {
            new CreateOrderDto
                { Amount = 100, CustomerName = "" },
            "CustomerName"
        },
    };

[Theory]
[MemberData(nameof(InvalidOrders))]
public void Validate_InvalidOrder_ReturnsCorrectError(
    CreateOrderDto dto, string expectedField)
{
    var result = _validator.Validate(dto);
    Assert.False(result.IsValid);
    Assert.Contains(expectedField, result.Errors[0]);
}

```

Async tests

```

[Fact]
public async Task GetByIdAsync_ExistingId_ReturnsOrder()
{
    // Arrange:
    var repo = new InMemoryOrderRepository();
    var created = await repo.CreateAsync(
        new Order { CustomerName = "Sathish", Amount = 500m });
    var service = new OrderService(
        repo, new FakeEmailService());

    // Act:
    var order = await service.GetByIdAsync(
        created.Id, CancellationToken.None);

    // Assert:
    Assert.NotNull(order);
    Assert.Equal("Sathish", order.CustomerName);
    Assert.Equal(500m, order.Amount);
}

```

Key Takeaways for 10.2

- xUnit discovers [Fact] (single test) and [Theory] (parameterised) automatically
- AAA pattern -- Arrange, Act, Assert -- the universal test structure
- Naming: MethodUnderTest_Scenario_ExpectedBehavior -- tests are documentation
- [Theory]+[InlineData] -- same logic with different inputs, no duplication
- Test methods can be async Task -- xUnit handles them correctly
- xUnit creates a NEW instance per [Fact] -- tests are isolated by default

Memory aid: xUnit tests are recipes. [Fact] is one recipe. [Theory] is a template where you substitute different ingredients (InlineData). AAA is the structure: gather ingredients (Arrange), cook (Act), taste (Assert).

Debugging Tips

Symptom	Cause	Fix
Test not discovered by test runner	Method not marked [Fact] or [Theory], or not public	Add [Fact], make method public, ensure class is public
async Task test hangs	Deadlock from .Result or .Wait() inside async test	Use await throughout; never block async code in tests
[Theory] test count is wrong	Missing [InlineData] attributes or wrong parameter count	Each [InlineData] must match the method parameter count exactly
Test passes in isolation but fails in batch	Shared static state between tests	Remove static fields from test class; use instance fields

10.3. Assertions and Test Lifecycle (Setup/Teardown)

Builds on: 10.2 (xUnit basics -- assertions used inside [Fact] tests), 1.13 (exception handling -- Assert.Throws tests exception paths).

xUnit assertions -- the complete reference

```
// Equality:
Assert.Equal(expected, actual);
Assert.NotEqual(expected, actual);
Assert.Equal(3.14, result, precision: 2);

// Null:
Assert.Null(value);
Assert.NotNull(value);

// Boolean:
Assert.True(condition);
Assert.False(condition);

// Collections:
Assert.Empty(collection);
Assert.NotEmpty(collection);
Assert.Single(collection);
Assert.Contains(item, collection);
Assert.DoesNotContain(item, collection);
```

```

Assert.Contains(collection,
    item => item.Amount > 100); // predicate
Assert.All(collection,
    item => Assert.True(item.Amount > 0));

// Type:
Assert.IsType<Order>(obj);
Assert.IsAssignableFrom<IOrder>(obj);

// Exceptions:
Assert.Throws<ArgumentException>(
    () => new OrderValidator().Validate(null!));
await Assert.ThrowsAsync<InvalidOperationException>(
    async () => await service.CancelAsync(id, ct));

// Exception message:
var ex = Assert.Throws<ArgumentException>(
    () => validator.Validate(badDto));
Assert.Contains("Amount", ex.Message);

// String:
Assert.StartsWith("Order", value);
Assert.Contains("customer", value);
Assert.Matches(@"^\d{4}-\d{2}-\d{2}$", dateStr);

// Range:
Assert.InRange(value, low: 1, high: 100);

```

Custom assertions -- extract repeated patterns

```

// Extract repeated multi-assertion patterns:
public static class OrderAssertions
{
    public static void IsValidPendingOrder(Order? order)
    {
        Assert.NotNull(order);
        Assert.Equal("Pending", order!.Status);
        Assert.True(order.Amount > 0,
            $"Amount should be positive: {order.Amount}");
        Assert.NotEmpty(order.Items);
    }
}

// Tests become cleaner:
var order = await service.CreateAsync(dto, ct);
OrderAssertions.IsValidPendingOrder(order);

```

Test lifecycle -- how xUnit creates test classes

xUnit creates a NEW instance of the test class for every [Fact]. This ensures test isolation -- no shared state between tests. The constructor runs before EACH test (equivalent to [SetUp] in NUnit). IDisposable.Dispose runs after each test (equivalent to [TearDown]).

```

public class OrderServiceTests
{

```



```

// Recreated fresh for EVERY single test:
private readonly InMemoryOrderRepository _repo;
private readonly FakeEmailService _email;
private readonly OrderService _service;

// Runs before each test:
public OrderServiceTests()
{
    _repo = new InMemoryOrderRepository();
    _email = new FakeEmailService();
    _service = new OrderService(_repo, _email);
}

[Fact]
public async Task Test1_IsolatedFreshRepo() { ... }
// _repo is empty, _service is brand new

[Fact]
public async Task Test2_AlsoIsolated() { ... }
// Different _repo instance -- Test1 data is gone
}

```

IDisposable -- cleanup after each test

```

public class DatabaseTests : IDisposable
{
    private readonly AppDbContext _context;
    private readonly SqlOrderRepository _repo;

    // Setup before each test:
    public DatabaseTests()
    {
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;
        _context = new AppDbContext(options);
        _repo = new SqlOrderRepository(_context);
    }

    [Fact]
    public async Task CreateAsync_PersistsOrder()
    {
        var order = await _repo.CreateAsync(
            new Order { CustomerName = "Sathish", Amount = 500m });
        var retrieved = await _repo.GetByIdAsync(order.Id);
        Assert.NotNull(retrieved);
    }

    // Cleanup after each test:
    public void Dispose() => _context.Dispose();
}

```

IClassFixture -- shared setup across all tests in a class

When setup is expensive (starting a test server, seeding a read-only database), share it across all tests with `IClassFixture`. The fixture is created ONCE for the entire test class.

```
// Fixture -- created once for the entire class:
public class OrderDatabaseFixture : IDisposable
{
    public AppDbContext Context { get; }

    public OrderDatabaseFixture()
    {
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase("SharedTestDb")
            .Options;
        Context = new AppDbContext(options);
        Context.Orders.AddRange(
            new Order { Id = 1, CustomerName = "Sathish" },
            new Order { Id = 2, CustomerName = "Priya" });
        Context.SaveChanges();
    }

    public void Dispose() => Context.Dispose();
}

// Test class receives the fixture via constructor:
public class OrderReadTests
    : IClassFixture<OrderDatabaseFixture>
{
    private readonly AppDbContext _context;

    public OrderReadTests(OrderDatabaseFixture fixture)
        => _context = fixture.Context;

    [Fact]
    public async Task GetAll_ReturnsSeededOrders()
    {
        var orders = await _context.Orders.ToListAsync();
        Assert.Equal(2, orders.Count);
    }
}
```

Key Takeaways for 10.3

- xUnit creates a new test class instance per `[Fact]` -- constructor = setup
- `IDisposable.Dispose` = teardown -- runs after each test
- No `[SetUp]/[TearDown]` attributes -- constructor and `IDisposable` are the equivalent
- `Assert.ThrowsAsync` for async exception testing -- not `Assert.Throws`
- `IClassFixture` shares expensive setup across all tests in the class
- Each test should be independent -- no test depends on another test's side effects

Memory aid: xUnit tests are hotel rooms. Each guest (test) gets a fresh room (new class instance). Constructor is housekeeping before check-in. Dispose is housekeeping after checkout. `IClassFixture` is the hotel building -- shared by all guests, built once.

Debugging Tips

Symptom	Cause	Fix
Assert.Equal fails on objects that look identical	Reference equality used -- two different object instances	Override Equals/GetHashCode or compare specific properties
Assert.ThrowsAsync not catching exception	Using Assert.Throws instead of Assert.ThrowsAsync for async	Use await Assert.ThrowsAsync(async () => await method())
State from Test1 affects Test2	Shared static state or singleton in test setup	Remove static state; use instance fields created in constructor
IClassFixture context shows stale data	Tests mutating shared fixture -- no isolation	Use constructor/IDisposable for mutable data; IClassFixture for read-only

10.4. Mocking (Moq)

Builds on: 9.5 (DI -- mocking replaces injected dependencies), 9.1 (DIP -- only interfaces and virtual methods can be mocked), 10.2 (xUnit -- mocks used inside tests), 6.1 (delegates -- Moq uses lambdas for setup).

What is mocking and why does it exist?

When testing OrderService, you don't want a real database or email server. Moq creates fake implementations of interfaces at test time, with full control over what they return and the ability to verify they were called.

```
// Add to test project:  
// <PackageReference Include="Moq" Version="4.20.*" />
```

Basic Moq -- Setup, Returns, Verify

```
using Moq;  
  
[Fact]  
public async Task GetByIdAsync_ExistingId_ReturnsOrder()  
{  
    // Arrange -- create mock objects:  
    var mockRepo = new Mock<IOrderRepository>();  
    var mockEmail = new Mock<IEmailService>();  
  
    var expectedOrder = new Order  
    { Id = 1, CustomerName = "Sathish", Amount = 500m };  
  
    // Setup -- configure what mock returns:  
    mockRepo  
        .Setup(r => r.GetByIdAsync(  
            1, It.IsAny<CancellationToken>()))  
        .ReturnsAsync(expectedOrder);  
  
    // .Object = the actual IOrderRepository instance:  
    var service = new OrderService(  
        mockRepo.Object, mockEmail.Object);  
  
    // Act:
```

```

var order = await service.GetByIdAsync(
    1, CancellationToken.None);

// Assert:
Assert.NotNull(order);
Assert.Equal("Sathish", order.CustomerName);

// Verify -- was repo called exactly once with id=1?
mockRepo.Verify(
    r => r.GetByIdAsync(
        1, It.IsAny<CancellationToken>()),
    Times.Once);
}

```

Argument matchers

```

// It.IsAny<T>() -- matches any value of type T:
.Setup(r => r.GetByIdAsync(
    It.IsAny<int>(), It.IsAny<CancellationToken>()))

// It.Is<T>(predicate) -- matches when predicate true:
.Setup(r => r.GetByIdAsync(
    It.Is<int>(id => id > 0),
    It.IsAny<CancellationToken>()))

// Exact value -- matches only that specific value:
.Setup(r => r.GetByIdAsync(
    42, It.IsAny<CancellationToken>()))

```

Returning values and throwing

```

// ReturnsAsync -- for async methods returning Task<T>:
mockRepo.Setup(r => r.GetByIdAsync(1, default))
    .ReturnsAsync(new Order { Id = 1 });

// Returns null:
mockRepo.Setup(r => r.GetByIdAsync(999, default))
    .ReturnsAsync((Order?)null);

// Throw an exception:
mockRepo.Setup(r => r.GetByIdAsync(
    It.IsAny<int>(), default))
    .ThrowsAsync(new SqlException());

// Different values on successive calls:
mockRepo.SetupSequence(r => r.GetByIdAsync(1, default))
    .ReturnsAsync(new Order { Id = 1 }) // first call
    .ReturnsAsync((Order?)null);      // second call

```

Verify -- asserting a mock was called

```

// Called exactly once:
mockEmail.Verify(
    e => e.SendConfirmationAsync(
        It.Is<Order>(o => o.Id == 1),

```

```

        It.IsAny<CancellationToken>()),
        Times.Once);

// Never called:
mockEmail.Verify(
    e => e.SendConfirmationAsync(
        It.IsAny<Order>(),
        It.IsAny<CancellationToken>()),
    Times.Never);

// Called exactly N times:
mockRepo.Verify(
    r => r.UpdateAsync(
        It.IsAny<Order>(), default),
    Times.Exactly(3));

// Ensure every Setup was called (strict mode):
mockRepo.VerifyAll();

```

Complete realistic test with Moq

```

public class OrderServiceTests
{
    private readonly Mock<IOrderRepository> _mockRepo;
    private readonly Mock<IEmailService> _mockEmail;
    private readonly OrderService _service;

    public OrderServiceTests()
    {
        _mockRepo = new Mock<IOrderRepository>();
        _mockEmail = new Mock<IEmailService>();
        _service = new OrderService(
            _mockRepo.Object, _mockEmail.Object);
    }

    [Fact]
    public async Task CreateAsync_ValidOrder_SavesAndEmails()
    {
        var savedOrder = new Order
        {
            Id = 1, CustomerName = "Sathish",
            CustomerEmail = "s@zoho.com", Status = "Pending"
        };

        _mockRepo
            .Setup(r => r.CreateAsync(
                It.IsAny<Order>(),
                It.IsAny<CancellationToken>()))
            .ReturnsAsync(savedOrder);

        var result = await _service.CreateAsync(
            new CreateOrderDto
            {
                CustomerName = "Sathish",
                Amount = 500m
            }
        );
    }
}

```

```

        }, CancellationTokens.None);

Assert.Equal(1, result.Id);
Assert.Equal("Pending", result.Status);

_mockRepo.Verify(
    r => r.CreateAsync(
        It.Is<Order>(o =>
            o.CustomerName == "Sathish"),
        It.IsAny<CancellationTokens>()),
    Times.Once);

_mockEmail.Verify(
    e => e.SendConfirmationAsync(
        It.Is<Order>(o => o.Id == 1),
        It.IsAny<CancellationTokens>()),
    Times.Once);
}

[Fact]
public async Task Cancel_ShippedOrder_DoesNotSave()
{
    _mockRepo
        .Setup(r => r.GetByIdAsync(1, default))
        .ReturnsAsync(new Order
            { Id = 1, UserId = 1, Status = "Shipped" });

    await Assert.ThrowsAsync<InvalidOperationException>(
        () => _service.CancelAsync(
            1, userId: 1, CancellationTokens.None));

    _mockRepo.Verify(
        r => r.UpdateAsync(
            It.IsAny<Order>(),
            It.IsAny<CancellationTokens>()),
        Times.Never);
}
}

```

Moq vs manual fakes

Use Moq when:

- > You need to verify mock was called (or not called)
- > Different return values needed per scenario
- > Interface has many methods -- full fake is tedious

Use manual fakes when:

- > Fake has meaningful logic (InMemoryRepository)
- > Many tests share the same fake
- > Fake needs to maintain state across calls

Key Takeaways for 10.4

- Moq creates fake implementations of interfaces dynamically at test time

- new Mock() creates the mock; .Object gets the interface instance
- Setup(r => r.Method(args)).ReturnsAsync(value) configures return values
- It.IsAny() matches any arg; It.Is(pred) matches conditionally
- Verify(r => r.Method(args), Times.Once) asserts the mock was called
- MockBehavior.Strict throws on any unmocked call -- catches unexpected deps

Memory aid: Moq is an actor agency. You say I need someone to play IOrderRepository. Moq sends an actor (mock). You give them a script (Setup): when asked for order 1, return this. Then verify they followed the script (Verify).

Debugging Tips

Symptom	Cause	Fix
Moq throws NotSupportedException on Setup	Method is not virtual or interface member	Only interfaces and virtual methods can be mocked with Moq
Verify throws MockException -- was never called	Method was called with different arguments than specified	Use It.IsAny() to broaden matching; log actual calls
Mock returns null when a value was expected	Setup not matched -- args differ from actual call	Check arg matchers; use It.IsAny() to debug first
ReturnsAsync not available	Method returns Task not Task<T>, use Returns(Task.CompletedTask)	For void-return async: .Returns(Task.CompletedTask)

10.5. Test-Driven Development (TDD) Concept

Builds on: 10.2-10.4 (xUnit and Moq -- TDD uses all of these), 9.1 (SOLID -- TDD naturally produces SOLID code).

What is TDD?

TDD is a development process where you write the TEST FIRST, watch it fail, then write the minimum code to make it pass. The cycle is Red-Green-Refactor.

```
RED      Write a failing test (defines desired behaviour)
GREEN    Write minimum code to make the test pass
REFACTOR Clean up without breaking the test
```

Repeat for every new behaviour.

TDD in practice -- worked example

Red -- write a failing test first:

```
[Fact]
public void Validate_NullDto_ThrowsArgumentNullException()
{
    var validator = new OrderValidator();
    Assert.Throws<ArgumentNullException>(
        () => validator.Validate(null!));
}
// This fails because OrderValidator doesn't exist yet
```

Green -- write minimum code to pass:

```
public class OrderValidator
{
    public ValidationResult Validate(CreateOrderDto? dto)
    {
        if (dto is null)
            throw new ArgumentNullException(nameof(dto));
        return ValidationResult.Ok();
    }
}
```

Red -- next test, write it first:

```
[Fact]
public void Validate_ZeroAmount_ReturnsAmountError()
{
    var dto = new CreateOrderDto
        { Amount = 0, CustomerName = "X" };
    var result = new OrderValidator().Validate(dto);
    Assert.False(result.IsValid);
    Assert.Contains("Amount", result.Errors[0]);
}
// Fails -- Validate currently always returns Ok
```

Green -- extend the implementation:

```
public ValidationResult Validate(CreateOrderDto? dto)
{
    if (dto is null)
        throw new ArgumentNullException(nameof(dto));

    var errors = new List<string>();
    if (dto.Amount <= 0)
        errors.Add("Amount must be positive");

    return errors.Any()
        ? ValidationResult.Fail(errors)
        : ValidationResult.Ok();
}
```

Refactor -- clean up repeated setup:

```
// After several tests, extract shared setup:
public class OrderValidatorTests
{
    private readonly OrderValidator _validator = new();

    [Fact] void Validate_NullDto_Throws() { ... }
    [Fact] void Validate_ZeroAmount_ReturnsError() { ... }
    [Fact] void Validate_EmptyName_ReturnsError() { ... }
}
```

What TDD produces naturally

TDD forces you to think about the public API before the implementation. This produces better interfaces (designed from consumer perspective), testable code (hard to test = fix the design first), SOLID code (testing complex classes is painful so you split them), and comprehensive test

coverage (every feature starts with a test).

When TDD works well vs when it doesn't

TDD works well for:

- > Business logic and validation rules
- > Algorithms and data transformations
- > Known requirements with clear acceptance criteria

TDD is harder for:

- > UI development (hard to test-drive visually)
- > Exploratory code -- requirements not yet clear
- > Prototypes and spike solutions

Pragmatic approach:

- > Use TDD for core business logic
- > Write tests alongside for infrastructure code
- > Never skip tests entirely

Key Takeaways for 10.5

- TDD = Red-Green-Refactor: failing test -> minimum code -> clean up
- Write the test first -- it defines desired behaviour before implementation
- TDD naturally produces testable SOLID code
- Each test covers one behaviour -- small, focused tests
- Not religious adherence -- use TDD where it adds value

Memory aid: TDD is like building a bridge by specifying the load test first, then building until it passes, then reinforcing the structure. You know exactly when it's strong enough -- when the test goes green.

10.6. Integration Testing Basics

Builds on: 10.2 (xUnit -- integration tests use the same framework), 10.3 (IClassFixture -- used for test server setup), 7.3 (async/await -- all integration tests are async).

Unit tests vs integration tests

Unit tests:

- > Test one class in isolation
- > All dependencies mocked or faked
- > Fast -- milliseconds per test
- > Many -- hundreds or thousands
- > Run on every save/commit

Integration tests:

- > Test multiple components working together
- > Real dependencies (DB, HTTP, DI container)
- > Slower -- seconds per test
- > Fewer -- tens or hundreds
- > Run on CI/CD pipeline

WebApplicationFactory -- the ASP.NET Core test server

WebApplicationFactory creates an in-memory version of your entire ASP.NET Core application -- real routing, real middleware, real DI container -- but no network calls.

```
// <PackageReference Include=
//   "Microsoft.AspNetCore.Mvc.Testing" Version="9.*" />

public class OrdersApiTests
    : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public OrdersApiTests(
        WebApplicationFactory<Program> factory)
    {
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task GetOrders_ReturnsOk()
    {
        // Real HTTP through real middleware:
        var response = await _client.GetAsync("/api/orders");
        response.EnsureSuccessStatusCode();
    }

    [Fact]
    public async Task CreateOrder_ValidDto_Returns201()
    {
        var dto = new CreateOrderDto
            { CustomerName = "Sathish", Amount = 500m };

        var response = await _client.PostAsJsonAsync(
            "/api/orders", dto);

        Assert.Equal(
            HttpStatusCode.Created, response.StatusCode);
        var created = await response.Content
            .ReadFromJsonAsync<OrderDto>();
        Assert.NotNull(created);
        Assert.Equal("Sathish", created!.CustomerName);
    }
}
```

Customising WebApplicationFactory -- swapping services

```
public class CustomWebApplicationFactory
    : WebApplicationFactory<Program>
{
    protected override void ConfigureWebHost(
        IWebHostBuilder builder)
    {
        builder.ConfigureServices(services =>
```

```

{
    // Remove the real DbContext registration:
    var descriptor = services.SingleOrDefault(
        d => d.ServiceType ==
            typeof(DbContextOptions<AppDbContext>));
    if (descriptor != null)
        services.Remove(descriptor);

    // Add in-memory database instead:
    services.AddDbContext<AppDbContext>(options =>
        options.UseInMemoryDatabase("TestDb"));

    // Seed test data:
    var sp = services.BuildServiceProvider();
    using var scope = sp.CreateScope();
    var context = scope.ServiceProvider
        .GetRequiredService<AppDbContext>();
    context.Orders.Add(new Order
    {
        Id = 1,
        CustomerName = "Sathish",
        Amount = 500m,
        Status = "Pending"
    });
    context.SaveChanges();
});
}
}

```

Key Takeaways for 10.6

- Integration tests verify multiple components working together
- WebApplicationFactory creates an in-memory ASP.NET Core test server
- Override ConfigureWebHost to swap SQL with in-memory database
- Use IClassFixture -- the factory is created once per test class
- Integration tests are slower -- run on CI, not on every keypress
- Strategy: many unit tests for business logic, fewer for API contracts

Memory aid: Unit tests test individual LEGO bricks. Integration tests snap the whole model together and check it looks right. WebApplicationFactory is the LEGO baseplate -- holds everything in place without needing a real table (production server).

Debugging Tips

Symptom	Cause	Fix
WebApplicationFactory cannot find Program class	Program.cs uses top-level statements -- class is internal	Add: public partial class Program { } at end of Program.cs
Integration test DbContext shows no data	Each test gets a fresh factory instance -- data not seeded	Use IClassFixture to share factory; seed data in fixture constructor
HttpClient returns 404 for all routes	Route prefix mismatch between test URL and controller route	Check [Route] attributes; ensure URL matches controller exactly

Symptom	Cause	Fix
Integration test passes locally but fails on CI	Time zone or locale differences, or missing env variables	Use UTC times; set required env vars in CI pipeline config

Section 10 -- Complete Key Takeaways

Topic	Core concept	Primary tool
-----+-----+-----		
10.1 Debugging	Pause, inspect, navigate	Breakpoints, Watch
10.2 xUnit	Discover and run tests	[Fact], AAA pattern
10.3 Assertions	Verify expected outcome	Assert.*, lifecycle
10.4 Moq	Replace with fakes	Mock<T>, Setup, Verify
10.5 TDD	Design through tests	Red-Green-Refactor
10.6 Integration	Test the whole system	WebApplicationFactory

.NET-specific rules:

- [Fact] for single tests, [Theory]+[InlineData] for parameterised
- Moq: only interfaces and virtual methods can be mocked
- Assert.ThrowsAsync<T> not Assert.Throws<T> for async exception testing
- WebApplicationFactory requires public partial class Program {} if internal
- EF Core in-memory DB: use Guid.NewGuid().ToString() as name for isolation

Section 10 complete. All 6 roadmap topics (10.1 through 10.6) covered. Say next to begin Section 11: Version Control (Git).